Ders 21 Genişletilmiş Notu: Birleştirme (Merge) Çözümlerinin İncelenmesi ve Nöbetçi Değer

Ders 21 Genişletilmiş Notu: Birleştirme (Merge) Çözümlerinin İncelenmesi ve Nöbetçi Değer

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kelime Okuma ve Karakter Dizisi İşlemlerini Kendimiz Yazmak
- 2. Bir Programın Analizi
- 3. Akış Diyagramı ve Kod İlişkisi
- 4. Matris Üzerinde Kelimelerle Çalışmak
- 5. İki Sıralı Diziyi Birleştirme: Birinci Çözümün İncelenmesi
- 6. İkinci Çözüm: Ayrı Sayaç Yerine C[I+J]
- 7. Eşitlik ve Kalan Elemanlar Üzerine Sorular
- 8. Sona Eleman Ekleme Fikri
- 9. Sona Hangi Değer Yazılmalı?
- 10. Döngü Koşulu: Küçük Eşit, AND ve OR
- 11. Merge Sort'un Genel Fikri ve Birleştirmenin Maliyeti
- Dersin Toparlaması

1. Kelime Okuma ve Karakter Dizisi İşlemlerini Kendimiz Yazmak

Dersin ilk bölümü, öğrencilerin matris üzerinde kelime işleyen bir programı yazarken sorduğu teknik sorulara ayrıldı. İlk konu okuma fonksiyonlarının seçimiydi. Boşluk içeren bir metin okunacaksa `gets` gerekir, çünkü bütün metni boşluklarıyla birlikte almak gerekir. Matrisin her satırına yalnızca tek bir kelime okunacaksa boşluk problemi yoktur. Bu durumda her satıra `scanf("%s", ...)` ile okuma yapmak daha uygundur. Hoca `scanf` ile `%s` kullanmanın ve `gets`'in ayrı bir hikâyesi olduğunu, bunu birinci sınıfta tartışmaya açmak istemediğini ve ileride görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** `gets` hedef dizinin boyutunu bilmediği için büyük olasılıkla taşma riski taşır ve C11 standardından (2011) çıkarılmıştır. `scanf` ile `%s` ise boşluk gördüğünde durur, bu yüzden tek kelime okumaya uygundur. Bu fark, hocanın "ayrı bir hikâye" dediği konunun büyük olasılıkla bir parçasıdır.

İkinci konu `string.h` kütüphanesinin neden kullanılmayacağıydı. Öğrencilerin aklına şu soru gelebilir: Kütüphaneler var, fonksiyonlar yazılmış, tekrar kullanılabilirlik önemli, neden yazılmış bir kodu yeniden yazıyoruz? Hocanın cevabı, bu fonksiyonların ileride sıklıkla kullanılacağı, ama birinci sınıfta işin nasıl yapıldığını öğrenmek için onları kendimizin yazmasının daha anlamlı ve sağlıklı olduğudur. Kendimizin yazması gereken işlemler şunlardır:

- Okunan kelimenin boyutunu bulmak, yani null karakterine kadar saymak.
- Karşılaştırma gerektiğinde iki kelimeyi harf harf karşılaştırmak ve karşılaştırmanın nasıl yapıldığını öğrenmek.

2. Bir Programın Analizi

Soru: Analiz, kodu çalıştırarak yapılabilir mi?

Cevap: Evet, yapılabilir. Kodu çalıştırıp farklı durumları göstermek yeterlidir. Fakat bir iki deneme yapıp analizi bitmiş saymak doğru değildir. Hemen hemen her türlü durum gösterilmelidir. Aynı tipte denemeler yapmak yerine farklı tipte denemeler gerekir. Hocanın kelime arama üzerinden verdiği örnekler şunlardır:

- Kelimenin bulunması ve bulunmaması.
- Kelimenin matrisin başında olması ve sonunda olması.

3. Akış Diyagramı ve Kod İlişkisi

Bir öğrencinin önce kodu yazıp sonra akış diyagramını çizeceğini söylemesi üzerine hoca bu alışkanlığı açıkça eleştirdi.

> **Hocanın vurgusu:** Önce kodu yazıp sonra akış diyagramı çizmek çok yanlış bir harekettir, buna alışmayın. Algoritmayı tasarlarken akış diyagramı çizmek mutlaka faydalıdır.

Hoca bunu kendi deneyimiyle destekledi. Birçok kişi "kodu yazmaya başladım ama bütünü oturtamadım, akış diyagramıyla her şey kolaylaştı" der. Hoca da bazı problemlerde hâlâ böyle yaptığını söyledi. İleride çok iyi olunduğunda, sözde kod (pseudocode) ya da kod yazarken sıkıntı yaşanmıyorsa akış diyagramı atlanabilir. Ama bu ilk sene yapılacak bir şey değildir.

Soru: Önce kabataslak bir akış diyagramı çizip, koda geçip, sonra akış diyagramını tamamlamak tavsiye edilir mi? Analiz kod üzerinde yapılınca daha kolay geliyor.

Cevap: Hoca bunu yasaklamadı, çalışma biçiminin kişiye bağlı olduğunu söyledi. Ama akışın temel olarak %80-90 tamamlanmış olması gerektiğini düşünüyor. Analiz kodda yapılınca ufak tefek sıkıntılar elbette çıkar ve orada düzeltilebilir. Bu kadar tamamlanmış bir akış diyagramında gözden kaçan yalnızca detaylar olur. Daha büyük bir sorun çıkıyorsa tasarım çok yanlış yapılmıştır ve bu da o zaman ortaya çıkar.

Soru: Kodda karakter dizisini null karakterine kadar okuyabiliyoruz. Akış diyagramında da böyle yapabilir miyiz?

Cevap: Evet. Dile özgü durumlar için endişelenmeye gerek yoktur. Akış diyagramı, kullanılan dilde yapılması gerekene uygun şekilde oluşturulur.

Soru: Akış diyagramında diziyi for döngüsü koymadan doğrudan "dizi okundu" diye almak sorun olur mu? Son derslerde öyle yapıyoruz.

Cevap: Hoca, okuma kısmında eski biçimi istemesinin sebebinin, dizinin eleman sayısını gösteren N ve M'nin ne anlama geldiğini hâlâ bilmeyen öğrencilerin olması olduğunu söyledi. Bu konu oturduktan sonra öğrenciler de hocanın kullandığı kısa okuma biçimini kullanabilecek.

4. Matris Üzerinde Kelimelerle Çalışmak

4.1 Sütun Sayısı ve Satırlardaki Kelimeler

Soru: Sütun sayısını da kullanıcıdan almalı mıyız?

Cevap: Hayır. Kaç kelime girileceği ayrı bir konudur, ama kelimelerin harf sayısı için kullanıcıdan bilgi almaya gerek yoktur. Sütun sayısı önceden tanımlanır.

Bir öğrenci, sütun sayısını en uzun kelimeye göre genişletmeyi önerdi. Fikri şuydu: Her kelime okunurken en büyük uzunluk bir değişkende tutulur, okuma ve yazma işlemleri bununla sınırlanır. Hoca önce, boyutun çalışma sırasında genişletilmesinin ancak dinamik bellek ayırmayla yapılabileceğini hatırlattı. Öğrenci diziyi yine sabit 100 sütunla açtığını, yalnızca okuma ve yazmayı sınırlamak istediğini söyleyince hoca buna gerek olmadığını belirtti.

Sebebi şudur: Her satırdaki kelime farklı sayıda sütun kaplar. Bir satırda kelime 3 sütun kaplar ve dördüncü sütunda null bulunur. Bir başkasında 5 sütun kaplar ve altıncı sütunda null bulunur. Kelimelerin aynı hizada olması gerekmez, zaten olmamalıdır. Bu yüzden bütün satırları en

uzun kelimeye göre genişletmek gerekmez. Hoca ayrıca, en uzun kelimeyi bulmak için ekstra döngü koyma gibi tavsiyelerin yanlış yönlendirebileceğini söyledi. Bunun gerçekten gerekip gerekmediğinin ayrıca düşünülmesini istedi.

4.2 Harfleri Başlangıç Noktasıyla Üretmek

Soru: Birden N'ye kadar tanımlanan bir for döngüsü gibi, A'dan Z'ye de tanımlanabilir mi? Hepsine tek tek atama mı gerekir?

Cevap: Tek tek atamaya gerek yoktur. Döngü 1'den 26'ya tanımlanıp bu değer A ile Z arasındaki geçiş olarak kullanılabilir. Harfleri elde etmek için başlangıç noktası olarak küçük `a` alınır. `a`'nın üzerine 0 eklenince `a`, 1 eklenince `b`, 2 eklenince `c` elde edilir.

4.3 Kelime Kelime ve Harf Harf Okumak

Soru: Akış diyagramında sütun sayısını bilmeden nasıl ilerleyeceğiz? Kelime kelime okuyabilir miyiz?

Cevap: Evet. Null görülen yere kadar bakılabilir. Matriste her satırda null görene kadar ilgili kelime alınır, karşılaştırma yapılır ve yola devam edilir. Özel bir durum için uzunluk bilgisi gerekebilir, ama okuma kısmı için buna ihtiyaç yoktur.

Bir öğrenci, kod gibi çalışan akış diyagramı programlarını hatırlattı. Soruyu soran öğrenci, bunları kastetmediğini, deftere yazsa bile çalışan bir akış diyagramı gibi düşündüğünü söyledi. Hoca her iki durumda da aynı şeyin geçerli olduğunu açıkladı: Kelime kelime okumak, her satırda belirli karakterleri okumaktır. `scanf` ile `%s`, karakterleri tek tek almak yerine hepsini birden okumayı sağlar. Fonksiyonun yaptığı tek iş budur. İstenirse aynı iş `%c` ile de yapılabilir: Matrisin `[0][0]` gözüne ilk kelimenin ilk harfi, `[0][1]` gözüne ikinci harfi, `[0][2]` gözüne üçüncü harfi okunur.

4.4 Matris Gösterimi

Akış diyagramında matris okurken indis adı olarak matrisle aynı harfi kullanmak uygun değildir. Matrisin adı büyük `A`, indisi küçük `a` olursa yazım karışır. Gösterim olarak çift köşeli parantez tercih edilir: `A[i][j]`. Tek parantez içinde virgülle ayırmak (`A[i, j]`) yanlış sayılmaz, ama tercih edilen ilk biçimdir.

4.5 Daha Az İş Yapan Çözümler

Soru: Matrisi 10 sütunlu açtık ama kelimeler 3 sütun kaplıyor. Yer değiştirmeden (swap) önce kelimenin uzunluğunu bulup ona göre dönmek, fazladan dönmekten daha mantıklı değil mi?

Cevap: Hoca bu özel soruya cevap vermedi, çünkü bunun öğrencinin kendi tasarım tercihi olduğunu söyledi. Genel ilkesi şudur: Daha iyi bir çözüme her zaman açıktır. Yer karmaşıklığı ya da işlem karmaşıklığı açısından daha az iş yaparak sonuca ulaşılıyorsa bu mutlaka belirtilmelidir.

5. İki Sıralı Diziyi Birleştirme: Birinci Çözümün İncelenmesi

Önceki derste kalınan yer merge sort'un birleştirme (merge) kısmıydı: Kendi içinde sıralı iki dizi, tek bir sıralı dizide birleştirilecek. Hoca öğrencilerden kendi akış diyagramlarını istedi, gelen bir çözümü ekranda açtı ve önce sınıfın incelemesini, sonra söz alarak yorum yapmasını istedi. Çözüm şu adımlardan oluşuyordu:

```
1. I = 1, J = 1
2. I < N ve J < M olduğu sürece:
     Dizi1[I] > Dizi2[J] ise:  result[K] = Dizi2[J],  J = J + 1
     değilse:                  result[K] = Dizi1[I],  I = I + 1
3. result dizisini yazdır
```

5.1 K Sayacının Sorunları

Sınıftan ilk gelen yorum, K'nın alınmasının unutulduğuydu. Hoca sorunun tanımlama olmadığını, başka bir şey olduğunu söyledi. Asıl eksikler şunlardı:

- **K'ya başlangıç değeri atanmamış.** I ve J ile birlikte `K = 1` ataması yapılmalıdır.
- **K hiç artırılmıyor.** Bir eleman yazıldıktan sonra K'nın bir ilerlemesi gerekir.

K'nın nerede artırılacağı özellikle soruldu. Bir öğrenci önce I ve J'nin artırıldığı yerde dedi. Hoca "her iki kolda da mı?" diye sordu, cevap kolların birleştiği yerdi.

> **Hocanın vurgusu:** Bir iş hem true kolunda hem false kolunda yapılacaksa iki tarafa da yazmak yerine, iş bittikten sonra kolların birleştiği yere bir kez yazılmalıdır. Burada `K = K + 1` böyle yazılır.

5.2 Kalan Elemanlar

İkinci eksik bir başka öğrenci tarafından yakalandı: Dizilerden biri bittiğinde diğerinde eleman kalmışsa bu elemanlar sonuca eklenmiyor. Döngü iki diziden biri bitince sona erer. Döngüden çıktıktan sonra hangi dizinin bitmediğine bakılır ve bitmeyen dizinin kalanı `result[K]`'dan başlayarak sona eklenir. Diziler sıralı olduğu için kalanlar doğrudan uca eklenebilir.

Bir öğrenci de bu konuda karıştırılan bir noktayı sordu: Dizi1 sürekli büyük gelirse `J` koşulu bozulur ve döngüden çıkılır, birleştirme yarım kalmaz mı? Hoca döngü koşulunda bir yanlışlık olmadığını söyledi. Yanlış olan, döngüden çıktıktan sonraki kısımdır. Orada hangi koşulun hâlâ doğru olduğuna göre diğer dizi sona eklenir.

Hocanın düzeltmeleriyle birinci çözüm şu hâle gelir:

```
1. I = 1, J = 1, K = 1
2. I < N ve J < M olduğu sürece:
     Dizi1[I] > Dizi2[J] ise:  result[K] = Dizi2[J],  J = J + 1
     değilse:                  result[K] = Dizi1[I],  I = I + 1
     K = K + 1      (iki kolun birleştiği yer)
3. Birinci dizi bitmediyse kalanını result'un sonuna ekle
4. İkinci dizi bitmediyse kalanını result'un sonuna ekle
5. result dizisini yazdır
```

> **Ders sonrası not:** Hoca bu koşulu "I küçük N, J küçük M" diye kabul etti. İndisler 1'den başlıyorsa son elemanlar da döngüde karşılaştırılmalıdır, bu yüzden koşul büyük olasılıkla `I <= N` ve `J <= M` olmalıdır. Kalanların kontrolü de aynı biçimde `I <= N` olur. `I < N` biçimi, ikinci çözümdeki gibi 0'dan başlayan indisler için doğrudur.

5.3 K Yerine I + J − 1 Kullanmak

Bir öğrenci kendi çözümünde K kullanmadığını, sonuç dizisinin indisi olarak `I + J - 1` yazdığını, fazladan değişken oluşturmadığı için bunun daha etkili olduğunu söyledi. Bir başka öğrenci, kalan elemanlar konusunda bunun yanlış olacağını öne sürdü. Ses sorunları nedeniyle bu tartışma tamamlanamadı. Hoca iki noktayı ayırdı.

Birincisi, bu yazımın çalışıp çalışmadığına önce bakmak gerektiğini söyledi. İkincisi, çalışsa bile daha avantajlı olduğu söylenemez: `K + 1` tek işlemdir, bir toplama yapılır. `I + J - 1` ise iki işlemdir, bir toplama ve bir çıkarma yapılır. Bu yüzden işlem açısından bir üstünlüğü yoktur.

> **Ders sonrası not:** Hoca çalışıp çalışmadığına bakılması gerektiğini söyleyip konuyu bıraktı. Rastgele sıralı dizilerle yapılan python denemesinde, kalan elemanlar da aynı indisle yazıldığında `I + J - 1` her durumda doğru konumu verdi. Bu yazım büyük olasılıkla doğrudur, itiraz edilen eksik yalnızca kalanların eklenmemesidir.

> **Hocanın vurgusu:** Bir değişken az kullanmak bu tür durumlarda özel bir kazanç sağlamaz. Gerçekten anlamsız, fazladan bir değişken tanımlıyorsanız gerek yoktur. Mümkünse başka değişken kullanmadan çözelim, ama bir değişken az kullanmak için büyük çaba harcamaya gerek yoktur.

5.4 Önceden Kontrol Eklemek

Soru: Döngüye girmeden önce bir kontrol koysak olur mu? İkinci dizinin ilk elemanı birinci dizinin en büyük elemanından bile büyükse dizileri doğrudan uç uca ekleyelim, ya da tersi.

Cevap: Bu çok sık görülen bir durum olsaydı yapılabilirdi. Ama hoca bunun için özel bir kontrol yapılmaması gerektiğini düşünüyor. Çünkü 10 seferden 1'inde ya da 100 seferden 7'sinde başımıza gelecek bir durum için her seferinde boşuna kontrol yapılmış olur.

> **Ders sonrası not:** Hoca bu durum için "99 kez boşuna kontrol" dedi. 100 seferden 7'sinde karşılaşılan bir durumda boşuna yapılan kontrol sayısı 93'tür.

6. İkinci Çözüm: Ayrı Sayaç Yerine C[I+J]

Aradan sonra başka bir öğrencinin çözümü açıldı. Bu çözümde indisler 0'dan başlıyor ve sonuç dizisine `C[I+J]` ile yazılıyor. Hoca önce şunu sordu: I ve J 1'den başlasa ilk eleman `C[2]`'ye yazılmaz mı? Öğrencinin cevabı, indislerin 0'dan başladığı oldu. İkisi de 0 iken ilk eleman 0. göze yerleşir, sonra indis birer ilerler. Bu mantıklıdır.

Çözümün ilk hâlinde döngü içinde iki ayrı kontrol vardı. Öğrenci de hoca da ikincisinin gereksiz olduğunu söyledi. Tek bir kontrol yeterlidir: True kolunda `B`'den, false kolunda `A`'dan eleman yerleştirilir. Düzeltilmiş çözüm şöyledir:

```
1. I = 0, J = 0
2. I < N ve J < M olduğu sürece:
     A[I] > B[J] ise:  C[I+J] = B[J],  J = J + 1
     değilse:          C[I+J] = A[I],  I = I + 1
3. I < N ise:
     K ← I || 1 || N−1  için  C[I+K] = A[K]    (birinci dizinin kalanı)
   değilse:
     K ← J || 1 || M−1  için  C[I+K] = B[K]    (ikinci dizinin kalanı)
```

Döngüden çıkınca `I < N` kontrolü yapılır. `I` hâlâ `N`'den küçükse birinci dizinin sonunda yerleştirilecek elemanlar vardır. Bu noktada `A` dizisinde artık `I` ile değil `K` ile ilerlenir ve `K`, `I`'dan başlar. Hoca buna özellikle dikkat çekti: Biraz önce `A` içinde `I` ile ilerliyorduk, şimdi `K` ile ilerliyoruz.

> **Ders sonrası not:** Hoca birinci dizinin kalanı için `C[I+K] = A[K]` yazımını okudu ve üzerinde durmadı. Bu noktada `I` sabit kalıp `K` da `I`'dan başladığı için doğru hedef büyük olasılıkla `C[K+J]`'dir. Dersteki `1 3 5 7 13` ve `2 4 6 8 9 10 11` dizileriyle python denemesinde `C[I+K]` yazımı 13'ü `C[11]` yerine `C[8]`'e yazıp 9'u siliyor. İkinci dalda `I = N` olduğu için `C[I+K] = B[K]` doğrudur.

`I < N` değilse ikinci dizi sonlanmamıştır. Hoca burada `J < M` diye ayrıca sormaya gerek olmadığını açıkladı. İkinci dizide hiçbir şey kalmamışsa, örneğin iki dizi aynı anda bittiyse, for döngüsü `K = J` ile `M - 1` aralığına hiç girmez. Bu yüzden for döngüsü doğrudan yazılabilir ve program tamamlanır.

> **Ders sonrası not:** Hoca "iki dizi aynı anda bittiyse" durumunu andı. Her adımda yalnızca bir indis ilerlediği için boş olmayan dizilerde bu durum büyük olasılıkla oluşmaz: 100.000 rastgele python denemesinde bir kez bile görülmedi. Hocanın vardığı sonuç değişmez, `J < M` kontrolü gerekmez.

7. Eşitlik ve Kalan Elemanlar Üzerine Sorular

7.1 Eşit Elemanlar Ne Olur?

Soru: Döngüdeki kontrolde eşitlik de olmalı değil mi? Elemanlar eşitse büyük de küçük de değildir, hiçbir şey yapılmıyor gibi görünüyor.

Cevap: Hoca bu yanlış düşüncenin konuşulmasının iyi olduğunu söyledi. Burada `A[I]`, `B[J]`'den büyük mü diye sorulur. Büyük değilse bu, küçük veya eşit demektir. Yani cevap hayır ise false koluna gidilir ve eşit eleman da yerleştirilir. Bu yüzden ayrı bir eşitlik kontrolüne gerek yoktur.

Hocanın vurgusu: Sorduğumuz sorunun karşılığı hayır ise, tersi geçerlidir. “Büyük mü?” sorusunun tersi “küçük eşit”, “küçük mü?” sorusunun tersi “büyük eşit”tir.

7.2 Kalanları Doğrudan Eklemek Sırayı Bozar mı?

Soru: Eleman sayısı az olan dizinin son elemanı büyük, eleman sayısı çok olan dizinin son elemanları küçük olsun. Uzun dizinin kalanını doğrudan sona eklersek, bu elemanlar büyük elemanın arkasına yazılmış olmaz mı? Bunun için ayrı kontrol gerekmez mi?

Cevap: Hoca bunu öğrencinin verdiği örnekle tahtada gösterdi. Diziler `1 3 5 7 13` ve `2 4 6 8 9 10 11` idi, yani kısa dizi 13 ile, uzun dizi 11 ile bitiyor.

Adım	I	J	A[I]	B[J]	Yazılan
1	0	0	1	2	1
2	1	0	3	2	2
3	1	1	3	4	3
4	2	1	5	4	4
5	2	2	5	6	5
6	3	2	7	6	6
7	3	3	7	8	7
8	4	3	13	8	8
9	4	4	13	9	9
10	4	5	13	10	10
11	4	6	13	11	11

11 yerleştirildikten sonra `J = M` olur ve döngüden çıkılır. `I < N` doğrudur, çünkü `I = 4` ve `N = 5`’tir. Birinci dizinin kalanı olan 13 sona konur ve algoritma biter. Sonuç `1 2 3 4 5 6 7 8 9 10 11 13` olur, sıra bozulmaz. Küçük elemanlar zaten 13’ten önce döngü içinde yerleşmiştir. Sona yalnızca hepsinden büyük olan kalan eklenir.

Öğrenci, karşılaştırmanın yalnızca en küçük eleman sayılı dizinin boyuna kadar gideceğini düşündüğü için böyle sormuştu. Hoca bunun doğru olmadığını söyledi: Döngü iki diziyi de izler, dizilerden biri tükenene kadar sürer.

8. Sona Eleman Ekleme Fikri

Daha önce soru çözülürken konuşulan bir fikre dönüldü: Dizilerin sonuna öyle bir eleman eklenebilir mi ki, ekstra bir for döngüsü ve if kontrolüyle “dizi bitti mi?” diye bakmak gerekmesin? Hoca iki dizi aldı: `1 3 5 8 9` ve `2 4 6 7 11 13`. Birini bilerek diğerinden uzun tuttu.

Bir öğrencinin önerisiyle iki dizinin sonuna da 1000 eklendi ve yerleştirme izlendi:

Karşılaştırma	Yazılan
1 ile 2	1
3 ile 2	2
3 ile 4	3
5 ile 4	4
5 ile 6	5
8 ile 6	6
8 ile 7	7
8 ile 11	8
9 ile 11	9
1000 ile 11	11
1000 ile 13	13
1000 ile 1000	yazılmaz, burada durulması gerekir

Her iki dizinin sonuna büyük bir sayı konduğu için dizilerden birinde eleman kalması sorunu ortadan kalktı. Bütün elemanlar tek bir döngü içinde, kontrolleri yapılarak yerleşti.

Soru: Böylece akış diyagramında neyden kurtulduk?

Cevap: Döngü sonrasında ekstra kontrol yapıp eklenmeyen dizinin kalanını eklemekten kurtulduk. Hoca bu kazancı abartmadı. Ortadan kalkan aslında bir tane if kontrolüdür. Bir for döngüsündeki kontrol de zaten bu kez döngü içindeki if ile yapılmış olur. Bu anlamda büyük bir değişiklik olmaz, ama ekstra bir kontrolden kurtulunur.

> **Ek bilgi (derste anlatılmadı):** Dizinin sonuna konan ve dizinin bittiğini anlamayı sağlayan bu değere büyük olasılıkla literatürde nöbetçi (sentinel) değer denir. Hoca derste bu adı kullanmadı. Nöbetçi için her dizide büyük olasılıkla fazladan bir göz ayrılması gerekir. 1000 gibi sabit bir değer de dizide daha büyük bir eleman varsa büyük olasılıkla işe yaramaz.

9. Sona Hangi Değer Yazılmalı?

Hoca, 1000 yerine ne yazılabileceğini sordu ve "1001, 1002" gibi bir cevap beklemediğini belirtti.

9.1 Bir Dizinin Son Elemanından Bir Fazlası

İlk öneri, dizinin son elemanından bir fazlasıydı: Dizi 13 ile bittiği için sona 14 yazılır. Aynı mantıkla diğer dizide 9'un sonuna 10 yazılması da konuşuldu, ama öğrenci bunun sorunu çözmeyeceğini fark edip vazgeçti. Bir başka öğrenci iki dizinin sonuna da 14 yazmayı önerdi.

> **Ders sonrası not:** Hoca her dizinin kendi son elemanının bir fazlasının sonucunu açıkça söylemedi. Dersteki dizilerle python denemesinde 9'dan sonra gelen 10, 11'den küçük olduğu için gerçek eleman gibi yazılıyor ve sonuçta 11 ile 13 kayboluyor. Bu öneri büyük olasılıkla yanlıştır.

Hoca buna karşı bir soru sordu: Birinci dizi 9 ile değil de 15 ile, yani 14'ten büyük bir sayıyla bitseydi ne olurdu? Bir öğrencinin 14'ün de yazılacağı yönündeki görüşü, başka bir öğrencinin 15'in 14'ten sonra geleceği yönündeki görüşüyle karşılaştı. Hoca tahminle olmayacağını söyleyerek iki dakika analiz süresi verdi: Herhangi bir dizinin son elemanından bir büyük olan sayı yazılırsa olur mu?

> **Ders sonrası not:** Hoca bu analizin sonucunu ayrıca söylemedi. `1 3 5 8 15` ve `2 4 6 7 11 13` dizilerinin sonuna 14 yazılınca python denemesi, 13'ten sonra 15 ile 14 karşılaştırıldığını ve 14'ün gerçek eleman gibi yazıldığını gösteriyor. Ardından ikinci dizinin indisi nöbetçiyi geçiyor ve 15 yazılamadan kalıyor. Bu öneri büyük olasılıkla bu dizilerde çalışmaz.

9.2 Büyük Olanın Bir Fazlası

Bir öğrenci iki dizinin son elemanlarını karşılaştırmayı, büyük olanın bir fazlasını diğer dizinin sonuna yazmayı önerdi. Hoca bunu `1 3 5 8 15` ve `2 4 6 7 11 13` dizileriyle denedi. 13 ile 15 karşılaştırılır, 15 büyüktür, 16 bulunur ve ikinci dizinin sonuna yazılır. Yerleştirme sırası şöyle ilerler: 1, 2, 3, 4, 5, 6, 7 ve 8 yerleşir. Sonra 15 ile 11 karşılaştırılıp 11, 15 ile 13 karşılaştırılıp 13 yazılır. 15 ile 16 karşılaştırılır, 15 yazılır ve bitilir. Bu çözüm çalışır, ama bir karşılaştırma gerektirir.

9.3 Karşılaştırmasız Çözüm: Son Elemanların Toplamı

Bu nedenle hoca yeni bir soru sordu: Hangisinin büyük olduğuna bakmadan, dizinin elemanlarını kullanarak iki dizinin sonuna bir şey koyabilir miyiz? Bir öğrencinin önerisi son elemanların toplamıydı: `1 3 5 8 15` ve `2 4 6 7 11 13` için $15 + 13 = 28$ iki dizinin de sonuna yazılır. Bir başka öğrenci, değerler negatif de olabileceği için mutlak değerin alınması gerektiğini ekledi: $|a_N| + |b_M|$.

> **Ders sonrası not:** Hoca toplam önerisini ve mutlak değer eklemesini kabul etti. Rastgele dizilerle yapılan python denemesinde, son elemanlardan biri 0 ise toplamın diğer dizinin son elemanına eşit çıkabildiği görüldü. Eşitlikte false koluna gidildiği için nöbetçi gerçek eleman gibi yazılabiliyor ve indis taşabiliyor. Bu yüzden nöbetçi

büyük olasılıkla $|a_N| + |b_M| + 1$ olmalıdır. Bu biçimle bütün denemeler doğru sonuç verdi. Derste kullanılan pozitif dizilerde bu sorun ortaya çıkmaz.

10. Döngü Koşulu: Küçük Eşit, AND ve OR

10.1 Yeni Örnek ve Yerleştirme

Hoca toplam fikrini yeni bir örnekle izledi: `2 4 6 8 12` ($N = 5$) ve `1 3` ($M = 2$). Son elemanların toplamı $12 + 3 = 15$'tir ve iki dizinin sonuna da 15 eklenir. İndisler 1'den başlar.

Adım	I	J	A[I]	B[J]	Yazılan
1	1	1	2	1	1
2	1	2	2	3	2
3	2	2	4	3	3
4	2	3	4	15	4
5	3	3	6	15	6
6	4	3	8	15	8
7	5	3	12	15	12

Bu noktada `I = 6`, `J = 3` olur. İki indis de kendi dizisinin nöbetçisini gösterir. Bütün gerçek elemanlar yerleşmiştir. Soru, döngü koşulunun ne olacağıdır.

Soru: `I < N` ve `J < M` kontrolü aynı kalır mı?

Cevap: Bir öğrenci önce aynı kalacağını söyledi, sonra bir fazlasının gerektiğini fark etti. Hoca, I ve J'nin 1'den başladığını hatırlattıktan sonra iki koşul sınıftan geldi.

10.2 Birinci Seçenek: I <= N+1 ve J <= M+1

Koşul `I <= N+1` ve `J <= M+1` olarak yazılırsa, `I = 6` ve `J = 3` iken ikisi de doğrudur, döngüye bir kez daha girilir. 15 ile 15 karşılaştırılır. Büyük olmadığı için false koluna gidilir ve 15 sonuca yazılır, `I` 7 olur. Koşul bozulduğu için döngüden çıkılır.

Sonuç dizisi `1 2 3 4 6 8 12 15` olur. Bir öğrencinin belirttiği gibi 15 diziye girse de sorun değildir. Çıktıda son eleman yazdırılmaz, ya da sonuç dizisinin boyutu bir küçültülür.

10.3 I + J ile Kontrol

Bir öğrenci döngüyü `I + J` üzerinden kontrol etmeyi önerdi: Bu örnekte `I + J < 8` denebilir, yani toplam indis `N + M` ile ilişkilendirilebilir. Hoca buna bakmak gerektiğini, olabileceğini söyledi ama diğer yorumlara geçti.

> **Ders sonrası not:** Hoca bu öneriyi kontrol etmeden bıraktı. Python denemesinde 1'den başlayan indislerle `I + J < 8` koşulu döngüyü `I = 5`, `J = 3` iken bitiriyor ve 12 yazılmıyor. Bütün elemanlar yazıldığında `I + J = 9` olduğu için koşul büyük olasılıkla `I + J < N + M + 2` olmalıdır. 0'dan başlayan indislerde bu `I + J < N + M` olur.

10.4 İkinci Seçenek: "Eşit Değil" Koşulu

Bir öğrenci farklı bir fikir getirdi: Büyüktür ve küçüktür yerine "eşit değil" koşulu kullanılsın. İki indis de son elemana geldiğinde döngüye girilmesin, böylece 15 hiç yazılmasın. Hoca bunu değişik ve güzel bir fikir olarak karşıladı, ama tahtaya yanlış anlayarak AND ile yazdı:

`I ≠ N+1 ve J ≠ M+1 olduğu sürece`

Bir öğrenci bunun doğru sonuç vereceğini söyledi. Bir başka öğrenci, `J` 3 olduğunda döngüye girilmeyeceğini söyledi. Hoca baştan izledi. 1, 2 ve 3 yerleşince `J = 3`, `I = 2` olur. `J = M + 1` olduğu için koşulun ikinci parçası yanlıştır, AND yüzünden döngü biter. Sonuçta yalnızca `1 2 3` yazılmıştır, 4, 6, 8 ve 12 yazılamadan kalır. Öğrencinin itirazı doğrudur.

Fikri öneren öğrenci, başta "veya" dediğini söyledi. Hoca bunu "ve" diye anladığını belirtti ve koşulu OR ile yeniden yazdı:

`I ≠ N+1 veya J ≠ M+1 olduğu sürece`

Şimdi `J = 3` iken `I ≠ 6` doğru olduğu için döngü sürer. 4, 6, 8 ve 12 yazılır. 12 yazılınca `I = 6` olur. $I = N + 1$ ve $J = M + 1$ olduğu için ikisi de yanlıştır ve döngüden çıkılır. Sonuç `1 2 3 4 6 8 12` olur, nöbetçilerden hiçbiri yazılmaz. Böylece ek bir yazdırma kısıtı da gerekmez.

> **Ek bilgi (derste anlatılmadı):** İki koşul arasındaki fark büyük olasılıkla De Morgan kuralıyla açıklanır. "İki indis de sona gelene kadar devam et" demek, $I = N + 1$ ve $J = M + 1$ durumunun değilini yazmaktır. Bu değil, $I \neq N + 1$ veya $J \neq M + 1$ olur. AND ile yazılan koşul ise indislerden biri sona gelir gelmez döngüyü bitirir.
>
> **Hocanın vurgusu:** Bu kadar detayı konuşmamız şunu gösterdi: Analizi açık ve düzgün yazmazsanız, kâğıt üzerine hızlıca karaladığınızda hatayı görme şansınız olmaz. Ayrıca her zaman AND koymak gerekmez, OR koşulu da olabilir. Sınır değerleri için büyüktür ve küçüktür yerine "farklı olma" koşulu da kullanılabilir. Bu değişik fikri zaman zaman uygulayabilirsiniz.

Hoca bu noktada döngüden çıktıktan sonraki kontrolde fazla iş yapılıp yapılmadığını da sormak istedi, ama bir öğrencinin başka sorusu üzerine bu konuya dönülmedi.

11. Merge Sort'un Genel Fikri ve Birleştirmenin Maliyeti

Soru: Bize diziler sıralı veriliyor. Rastgele bir diziyi önce ikiye bölüp sıralayıp sonra mı merge sort'a sokuyoruz? İlk başta iki sıralı dizi mi yapmamız gerekiyor?

Cevap: Hoca bunun şu anki konu olmadığını, hatta bu senenin konusu olmadığını, ama merak edildiği için özetleyeceğini söyledi. Merge sort böl ve yönet (divide and conquer) yaklaşımıyla çözülür. Sırasız dizi her turda ikiye bölünür. Bir eleman kalana, yani en küçük yapı taşına kadar bölme sürer. Bir eleman kalınca birleştirmeye başlanır ve bölünen parçalar yukarı doğru sıralı olarak birleştirilip bütün oluşturulur. Hoca merge sort'un dersin beş dakikasında anlatılacak bir konu olmadığını, öğrencilerin paylaşılan video ve kaynaklardan bakabileceğini söyledi.

Derste merge sort algoritmasının yalnızca merge kısmı yapıldı: İki sıralı parçanın birleştirilmesi. Bu iş için birkaç farklı yöntem kullanıldı ve hepsinin kendi artıları ve eksileri vardır. Kodu kısaltmak için son yöntem kullanıldı, bu da doğrudur.

> **Hocanın vurgusu:** Burada önemli olan, birleştirme işleminin (merge operation) karşılaştırma adımlarının $N + M$'de sonuçlanmasıdır. İki dizi uç uca eklenip sıralama yoluna gidilseydi oldukça maliyetli bir iş yapılmış olurdu. Karşılıklı elemanlar karşılaştırılıp sırayla yerleştirilerek iş kısaltılmıştır.
>
> **Ders sonrası not:** Hoca $N+M$ dedi. Bu büyük olasılıkla bir üst sınır olarak okunmalıdır. Python denemesinde nöbetçisiz döngüde karşılaştırma sayısı en fazla $N+M-1$ çıktı, çünkü son kalan eleman karşılaştırmasız eklenir. OR koşullu nöbetçili döngüde ise her gerçek eleman için bir karşılaştırma yapıldığından sayı tam $N + M$ oldu.
>
> **Ek bilgi (derste anlatılmadı):** Bu birleştirme adımı her bölünme seviyesinde tekrar edildiği için merge sort'un toplam karmaşıklığı büyük olasılıkla $O(n \log n)$'dir. Eleman sayısı n iken dizi yaklaşık $\log_2 n$ seviyede ikiye bölünür. Her seviyedeki birleştirmeler toplamda yaklaşık n iş yapar.

Dersin Toparlaması

Bu derste iki sıralı diziyi birleştirme problemi öğrenci çözümleri üzerinden incelendi. Sonuç dizisi indisine başlangıç değeri verilmesi ve artırma adımının iki kolun birleştiği yere konması görüldü. Dizilerden biri bitince diğerinin kalanının sona eklenmesi gerektiği gösterildi. `A[I] > B[J]` sorusunun hayır cevabının eşitliği de kapsadığı ve bu yüzden ayrı bir kontrol gerekmediği konuşuldu. Ardından kalan eleman kontrolünden kurtulmak için dizilerin sonuna büyük bir değer ekleme fikri işlendi ve bu değerin nasıl seçileceği tartışıldı. Hoca üç yolu özetledi: kalan elemanları döngüden sonra eklemek, son elemanların toplamını ekleyip küçük eşit koşuluyla

döngüyü kurmak ve son elemanı yazdırmamak, “eşit değil” koşulunu OR ile bağlayıp hiçbir nöbetçiyi yazmamak.

Hoca öğrencilerin iki sıralı diziyi birleştiren kodu yazıp doğru çalışıp çalışmadığını kontrol etmelerini önerdi. Sonraki derslerde algoritmalara biraz ara verilip işletim sistemi, ağ (network) ve belki işlemci mimarisi hakkında bilgiler konuşulacak. Ardından daha önce görülen aramanın daha etkili bir hâli olan ikili arama (binary search) algoritması işlenecek.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.